

Fire and Smoke Detection System — Complete Technical Report

Project: Real-Time Fire and Smoke Detection using Deep Learning

Type: Independent Research Project

Author: vipul1029 (tovipul.kr@gmail.com)

Report Date: 2026-09-11

Repository Branch: main

Table of Contents

- 1. Project Overview
- 2. Repository Structure
- 3. Technology Stack and Dependencies
- 4. Model Architecture and Weights
- 5. Training Configuration (From Checkpoint)
- 6. Training Performance Metrics
- 7. Source Code Deep Analysis
 - src/__init__.py
 - src/detector.py
 - src/pipeline.py
 - src/download_weights.py
 - run.py
- 8. Data Flow and Execution Model
- 9. Detection Output and Annotation System
- 10. Severity Scoring System
- 11. CLI Interface
- 12. Library API
- 13. Model Weight Management
- 14. Dataset Information
- 15. Configuration and Environment
- 16. Git History and Development Timeline
- 17. File Inventory
- 18. Known Issues and Discrepancies
- 19. Summary of Verified Facts

1. Project Overview

This is an AI system that detects fire and smoke in real time using a YOLOv8 deep learning model packaged in a clean Python application. It exposes three operating modes:

- **Image mode** — process a single image file, save annotated result
- **Video mode** — process a video file frame-by-frame, produce annotated output video
- **Webcam mode** — live continuous detection from a camera device

This is an independent research project focused on real-time fire and smoke detection using deep learning.

The system detects two classes:

Class ID	Label
0	smoke
1	fire

2. Repository Structure

```
fire and smoke detection/
■■■ src/                                # Python package
■   ■■■ __init__.py                    # Public API surface (exports 3 names)
■   ■■■ detector.py                   # YOLOv8 model loading and raw inference
■   ■■■ pipeline.py                   # FPS tracking, annotation, severity scoring
■   ■■■ download_weights.py           # Weight download and ensure helpers
■   ■■■ weights/                      # Created at runtime; stores .pt model file
■       ■■■ fire_detection_yolov8m.pt (gitignored; ~200 MB)
■■■ run.py                            # CLI entry point
■■■ requirements.txt                  # Pinned Python dependencies (5 entries)
■■■ README.md                        # User documentation
■■■ .gitignore                       # Excludes weights, venv, test artifacts
■■■ .gitattributes                   # Line-ending and binary rules
■■■ .venv/                           # Local virtual environment (gitignored)
■■■ DATASET_IDENTIFICATION_REPORT.md # Deep analysis of model checkpoint metadata
■■■ PRESENTATION_CONTENT_AND_SOURCES.md
■■■ SECTION_3_PROJECT_METHODODOLOGY.md
■■■ Fire_Smoke_Detection_Research_Presentation.pptx
■■■ Fire_Smoke_Detection_Research_Presentation_FINAL.pptx
■■■ gen_final.py                     # Presentation generation script
■■■ generate_presentation.py          # Presentation generation script
■■■ test_image.jpg                   # Sample input (gitignored after latest commit)
■■■ inputvideo.mp4                   # Sample input video (gitignored)
■■■ output.jpg                       # Sample output image (gitignored)
■■■ output.mp4                       # Sample output video (runtime-generated)
■■■ result.mp4                       # Sample annotated result (gitignored)
```

The `src/weights/` directory and all `.pt` files are gitignored. The model is downloaded automatically on first run.

3. Technology Stack and Dependencies

requirements.txt (pinned versions)

Package	Pinned Version	Role
ultralytics	8.2.18	YOLOv8 inference framework
numpy	1.26.4	Numerical array operations
opencv-python	4.9.0.80	Image/video I/O and drawing
requests	(latest)	HTTP download of model weights
torch	(latest)	PyTorch deep learning backend

Runtime Environment

- **Python version required:** 3.9+
- **OS:** Windows, macOS, or Linux (cross-platform)
- **Hardware:** CPU (default) or CUDA-capable NVIDIA GPU
- **Disk space:** ~200 MB for model weights
- **Virtual environment:** `.venv/` (Python 3.10 detected in `.venv/pyvenv.cfg`)

Installed venv packages (notable)

The virtual environment contains a full scientific Python stack installed by pip, including: `scipy`, `sympy`, `matplotlib`, `networkx`, `fonttools`, `tqdm`, `psutil`, `py-cpuinfo`, `pillow`, `pytz`, `tzdata`, `pyyaml`, `six`, `colorama`, `certifi`, `charset-normalizer`, `idna`, `urllib3`.

4. Model Architecture and Weights

Critical Correction: The Weights File Is Mismatched

The weights file is named `fire_detection_yolov8m.pt` (the "m" implies medium), but deep inspection of the checkpoint via `torch.load()` confirms the actual architecture is **YOLOv8n (nano)**.

Property	Value	Verified From
Weights filename	<code>fire_detection_yolov8m.pt</code>	filesystem
Actual architecture	YOLOv8n (NANO)	<code>model.yaml['depth_multiple']=0.33, ['width_multiple']=0.25</code>
Total parameters	3,011,238 (~3M)	<code>model.info()</code>
Number of layers	226	<code>model.info()</code>
Input resolution	640 × 640 pixels	<code>train_args['imgsz']</code>
Number of classes	2	<code>model.nc, model.yaml['nc']</code>
Class names	<code>{0: 'smoke', 1: 'fire'}</code>	<code>model.names</code>
Training run name	<code>yolov8n-dfire</code>	<code>train_args['name']</code>
Ultralytics training version	8.4.51	<code>ckpt['version']</code>
Ultralytics inference version	8.2.18	<code>requirements.txt</code>
Training date	2026-05-20	<code>ckpt['date']</code>
Training device	Apple Silicon (mps)	<code>train_args['device']</code>

YOLOv8 Variant Comparison

Variant	depth_multiple	width_multiple	~Parameters
n (nano) — THIS MODEL	0.33	0.25	~3M
s (small)	0.33	0.50	~11M
m (medium)	0.67	0.75	~25M

Variant	depth_multiple	width_multiple	~Parameters
l (large)	1.00	1.00	~44M
x (xlarge)	1.00	1.25	~68M

The README incorrectly states "YOLOv8 medium". The architecture is definitively YOLOv8n.

Weight Source

The model weights are hosted on Hugging Face:

`https://huggingface.co/rabahdev/fire-smoke-yolov8n/resolve/main/best.pt`

The local path where they are saved: `src/weights/fire_detection_yolov8m.pt`

5. Training Configuration (From Checkpoint)

All values below were extracted from the embedded `train_args` inside `src/weights/fire_detection_yolov8m.pt`.

Core Training Hyperparameters

Parameter	Value
Task	detect
Mode	train
Epochs	50
Batch size	16
Image size	640 × 640
Optimizer	auto (Ultralytics auto-selection)
Initial learning rate (lr0)	0.01
Final LR ratio (lrf)	0.01
Momentum	0.937
Weight decay	0.0005
Warmup epochs	3.0
Warmup momentum	0.8
Warmup bias LR	0.0
Early stopping patience	15 epochs
Seed	0
Fraction of dataset used	1.0 (100%)
Pretrained	True (transfer learning)
AMP (mixed precision)	Enabled
Training NMS IoU	0.7

Parameter	Value
Inference NMS IoU	0.45
Inference confidence threshold	0.25

Loss Weights

Parameter	Value
Box loss weight	7.5
Classification loss weight	0.5
DFL loss weight	1.5

Data Augmentation Pipeline

Augmentation	Value
Mosaic	1.0 (disabled for final 10 epochs)
Close mosaic (last N epochs)	10
Auto-augment strategy	randaugment
Random erasing	0.4
HSV hue jitter	0.015
HSV saturation jitter	0.7
HSV value jitter	0.4
Horizontal flip probability	0.5
Vertical flip probability	0.0
Translation	0.1
Scale	0.5
Shear	0.0
Rotation (degrees)	0.0
Perspective	0.0
Mixup	0.0
Copy-paste	0.0

6. Training Performance Metrics

Extracted from `train_metrics` inside the checkpoint.

Final Metrics at Epoch 50

Metric	Value
mAP@0.5	0.76498 (76.5%)

Metric	Value
mAP@0.5:0.95	0.44460 (44.5%)
Precision	0.77529 (77.5%)
Recall	0.69452 (69.5%)
val/box_loss	1.09252
val/cls_loss	1.25522
val/df_l_loss	0.99099

Training Progression (Selected Epochs)

Epoch	mAP@0.5	mAP@0.5:0.95	Precision	Recall
1	0.38729	0.17637	0.48093	0.42605
5	0.46727	0.21607	0.50985	0.47480
10	0.61737	0.32478	0.62633	0.58352
15	0.66175	0.35837	0.66454	0.61095
20	0.70351	0.39116	0.71708	0.63205
25	0.72236	0.40810	0.73811	0.64288
30	0.74310	0.42226	0.73695	0.67635
35	0.74924	0.43126	0.74940	0.68639
40	0.75766	0.44158	0.75997	0.68544
45	0.76240	0.44150	0.76756	0.68674
50	0.76498	0.44460	0.77529	0.69452

The model shows consistent monotonic improvement across all 50 epochs, with the largest gains in the first 20 epochs (mAP@0.5 going from 0.387 → 0.704) and a plateau forming after epoch 40.

7. Source Code Deep Analysis

src/ __init__.py

File: src/ __init__.py

Lines: 8

Role: Public API surface for the src package.

Exports exactly three names:

```
from .detector import FireDetector, Detection
from .pipeline import FireAIPipeline

__all__ = ["FireDetector", "Detection", "FireAIPipeline"]
```

This is the minimal, correct package init that any downstream import consumer would use.

src/detector.py

File: `src/detector.py`

Lines: 175

Role: Owns the YOLO model lifecycle — loading, weight management, inference, and the `Detection` result dataclass.

PyTorch Compatibility Patch (lines 15–26)

The file monkey-patches `torch.load` at module import time to force `weights_only=False`:

```
original_load = torch.load
def patched_load(*args, **kwargs):
    kwargs["weights_only"] = False
    return original_load(*args, **kwargs)
torch.load = patched_load
```

This is a backward-compatibility workaround for PyTorch 2.6+, which changed the default of `weights_only` to `True`, breaking loading of Ultralytics `.pt` files that embed non-tensor Python objects in the checkpoint. The patch ensures the model loads correctly regardless of the installed PyTorch version.

Constants

```
WEIGHTS_DIR = Path(__file__).parent / "weights" # src/weights/
FIRE_WEIGHTS = WEIGHTS_DIR / "fire_detection_yolov8m.pt" # full path to .pt file
```

Detection Dataclass

```
@dataclass
class Detection:
    bbox: List[int] # [x1, y1, x2, y2] in pixel coordinates
    confidence: float # 0.0 - 1.0
    class_id: int = 0 # 0=smoke, 1=fire
    label: str = "fire"
    crop: Optional[np.ndarray] = field(default=None, repr=False)

    @property
    def width(self) -> int: # x2 - x1
    @property
    def height(self) -> int: # y2 - y1
```

`crop` is excluded from `__repr__` to avoid spamming logs with numpy array data. The `bbox` is always clamped to frame boundaries before construction (lines 144–148).

FireDetector Class

Constructor parameters:

Parameter	Type	Default	Description
<code>conf_threshold</code>	float	0.25	Minimum confidence to keep a detection
<code>iou_threshold</code>	float	0.45	NMS IoU threshold

Parameter	Type	Default	Description
<code>input_size</code>	int	640	YOLO input resolution (pixels)
<code>device</code>	str	"cpu"	Inference device: "cpu", "cuda", "mps", etc.
<code>weights_url</code>	str or None	None	Override Hugging Face download URL

Initialization flow:

1. Stores all parameters.
2. Resolves `weights_url` — falls back to `FIRE_MODEL_HF_URL` env var, then to the hardcoded Hugging Face URL.
3. Creates `WEIGHTS_DIR` if it doesn't exist.
4. Calls `_load_model()`.

`_load_model()` logic (lines 76–94):

1. If `FIRE_WEIGHTS` does not exist → calls `_download_weights()`.
2. If `FIRE_WEIGHTS` still does not exist → sets `backend="unavailable"`, `available=False`, and returns (graceful degradation).
3. If weights exist → `from ultralytics import YOLO; self.model = YOLO(str(FIRE_WEIGHTS))`.
4. Reads `model.names` from the YOLO object to populate `self.class_names`.
5. Sets `self.available = True`.

`download_from_hf()` static method (lines 103–119):

Downloads weights via streaming HTTP with a 30-second timeout. Writes in 8 KB chunks. If any exception occurs, deletes the partial file and re-raises as `RuntimeError`. This prevents corrupted partial downloads from being silently used on subsequent runs.

`detect()` method (lines 121–162):

Calls `self.model.predict()` with:

- `stream=False` — returns all results at once
- `verbose=False` — suppresses Ultralytics console output

For each bounding box in results:

1. Extracts `xyxy` coordinates as integers.
2. Gets confidence and `class_id`.
3. Resolves label from `self.class_names`.
4. Clamps coordinates to `[0, frame_width] × [0, frame_height]`.
5. Skips degenerate boxes where `x2 <= x1` or `y2 <= y1`.
6. Crops the detected region from the frame.
7. Appends a `Detection` object.

Returns `List[Detection]` — empty list if model is not loaded.

`info` property (lines 164–174):

Returns a diagnostic dict with backend, model name, weights path, availability, and threshold values.

File: `src/pipeline.py`

Lines: 146

Role: Orchestrates detection, tracks per-frame performance metrics, renders visual annotations, and computes severity scores.

FireAIPipeline Class

Constructor parameters:

Parameter	Type	Default	Description
<code>detector_conf</code>	float	0.25	Passed to <code>FireDetector</code>
<code>device</code>	str	"cpu"	Passed to <code>FireDetector</code>
<code>target_fps</code>	int	5	Target processing frame rate
<code>on_results</code>	Callable or None	None	Optional callback invoked after every frame

Internal state:

Attribute	Type	Purpose
<code>frame_idx</code>	int	Monotonically increasing frame counter
<code>fps</code>	float	Rolling FPS computed over last 30 frames
<code>total_detected</code>	int	Cumulative count of all detections across all frames
<code>_fps_times</code>	list[float]	Timestamps of last 30 processed frames
<code>_latest_detections</code>	List[Detection]	Most recent frame's detections

`process_frame()` ***method (lines 39–62)***

- 1. Increments `frame_idx`.
- 2. Calls `self.detector.detect(frame)` — the only inference call.
- 3. Appends current timestamp to `_fps_times`; keeps only the most recent 30.
- 4. Computes rolling FPS: `count / elapsed` over the 30-frame window.
- 5. Invokes `on_results(detections, frame)` if a callback is registered.
- 6. Logs at DEBUG level: frame number, processing time, detection count.
- 7. Returns `List[Detection]`.

`annotate_frame()` ***method (lines 64–88)***

Imports `cv2` lazily (inside the function body) — if OpenCV is not installed, the method returns the original frame unchanged.

For each detection:

- Draws a rectangle: **red** (0, 0, 255) for fire, **orange** (0, 165, 255) for smoke.
- Measures text label with `cv2.getTextSize()` to size the background chip.
- Draws a black filled rectangle behind the label for contrast.
- Writes label text: e.g. "fire 85%" (confidence shown as integer percent).

After all detections, draws HUD text at the bottom-left:

```
FPS:24 Fire:1 Smoke:0 Model:fire-detection
```

HUD color logic:

- Red if any fire detected
- Orange if smoke only
- Green if nothing detected

`get_stats()` *method (lines 90–134)*

Returns a comprehensive statistics dictionary:

```
{
    "fire_count": int,
    "smoke_count": int,
    "total_hazards": int,
    "detections": [
        {
            "id": int,          # 1-indexed
            "bbox": [x1,y1,x2,y2],
            "label": str,
            "confidence": float # rounded to 3 decimal places
        },
        ...
    ],
    "total_potholes": 0,      # compatibility shim (always 0)
    "fps": float,
    "frame_count": int,
    "avg_confidence": float,  # 0.0 if no detections
    "severity_score": float,  # 0.0 - 100.0
    "severity_label": str,    # "low" | "medium" | "high" | "critical"
    "model": "fire-detection",
    "detector": dict,         # FireDetector.info
    "pipeline_type": "fire",
    "target_fps": int,
}
```

Note: `total_potholes: 0` is a compatibility shim suggesting this pipeline shares an API contract with a separate pothole detection pipeline.

`info` *property (lines 136–145)*

Returns pipeline metadata — detector info, tracker (none), inference backend (none), model name, pipeline type, and target FPS.

`src/download_weights.py`

File: `src/download_weights.py`

Lines: 47

Role: Standalone weight management utilities; can be run directly with `python -m src.download_weights`.

Module-level side effect at import: Calls `_load_env_file()` which reads a `.env` file from the project root (if present) and injects any found `KEY=VALUE` pairs into `os.environ`. This is done only for keys not already set in the environment (safe, non-overriding).

Functions:

Function	Signature	Behavior
<code>download_weights</code>	<code>(weights_url=None) -> Path</code>	Always downloads (re-downloads if present)
<code>ensure_weights</code>	<code>(weights_url=None) -> Path</code>	Downloads only if file is absent; idempotent

Both resolve the URL from: argument → `FIRE_MODEL_HF_URL` env var → hardcoded Hugging Face URL.

When run as `__main__`, calls `ensure_weights()` and prints the output path.

run.py

File: `run.py`

Lines: 99

Role: CLI entry point. Parses arguments, instantiates the pipeline, and dispatches to the appropriate processing function.

Three operating functions

`process_image(pipeline, source_path, output_path)` (lines 9–19):

1. `cv2.imread()` — exits with error if unreadable.
2. `pipeline.process_frame(frame)` → detections.
3. `pipeline.annotate_frame(frame, detections)` → annotated frame.
4. `cv2.imwrite(output_path, annotated)`.

`process_video(pipeline, source_path, output_path)` (lines 21–53):

1. Opens with `cv2.VideoCapture`.
2. Reads `CAP_PROP_FRAME_WIDTH`, `CAP_PROP_FRAME_HEIGHT`, `CAP_PROP_FPS`.
3. Creates `cv2.VideoWriter` with `mp4v` codec.
4. Frame loop uses `cap.grab()` + `cap.retrieve()` (two-step retrieval — more efficient than `cap.read()` for skipping frames).
5. Logs progress every 30 frames.
6. Releases both capture and writer on completion.

`process_webcam(pipeline, camera_index=0)` (lines 55–75):

1. Opens `cv2.VideoCapture(camera_index)`.
2. Runs an infinite loop: read → detect → annotate → `cv2.imshow`.
3. Exits when `q` is pressed (`cv2.waitKey(1) & 0xFF == ord('q')`).
4. Releases capture and destroys windows.

CLI argument dispatch (lines 77–98)

```
--source webcam      → process_webcam()
--source *.mp4/avi/mov/mkv → process_video()
--source <anything else> → process_image()
```

The output path defaults smart: if source is video but `--output` was not set (still at default `"output.jpg"`), the code switches the output to `"output.mp4"`.

Pipeline is created once and shared across the entire run:

```
pipeline = FireAIPipeline(detector_conf=args.conf, device=args.device, target_fps=30)
```

Note: `target_fps=30` is hardcoded in `run.py` — the `--fps` flag is not exposed in the CLI.

8. Data Flow and Execution Model

```
User Input (image / video file / webcam)
  ▼
run.py (CLI)
  ▼
  ■■■■ cv2.VideoCapture / cv2.imread
  ▼
FireAIPipeline.process_frame(frame: np.ndarray)
  ■
  ■■■■ updates frame_idx, timestamps, rolling FPS
  ■
  ■■■■ FireDetector.detect(frame)
  ■       ■
  ■       ■■■■ model.predict(source=frame, imsz=640, conf=0.25, iou=0.45)
  ■       ■       [YOLOv8n inference via Ultralytics]
  ■       ■
  ■       ■■■■ returns List[Detection]
  ■           each Detection has: bbox, confidence, class_id, label, crop
  ■
  ■■■■ optionally calls on_results(detections, frame)
  ■
  ■■■■ returns List[Detection]
  ▼
FireAIPipeline.annotate_frame(frame, detections)
  ■
  ■■■■ draws bounding boxes (red=fire, orange=smoke)
  ■■■■ draws label chips with confidence %
  ■■■■ draws HUD overlay (FPS, counts, model name)
  ■■■■ returns annotated frame (np.ndarray)
  ▼
Output: cv2.imwrite / cv2.VideoWriter / cv2.imshow
```

9. Detection Output and Annotation System

Detection Object Fields

Field	Type	Description
<code>bbox</code>	<code>List[int]</code>	<code>[x1, y1, x2, y2]</code> pixel coordinates
<code>confidence</code>	<code>float</code>	Model confidence, 0.0–1.0
<code>class_id</code>	<code>int</code>	<code>0</code> for smoke, <code>1</code> for fire
<code>label</code>	<code>str</code>	"smoke" or "fire"

Field	Type	Description
<code>crop</code>	<code>np.ndarray</code>	Pixel-cropped region from the frame
<code>.width</code>	<code>int</code> (property)	<code>x2 - x1</code>
<code>.height</code>	<code>int</code> (property)	<code>y2 - y1</code>

Visual Annotation Colors (BGR)

Class	Color Name	BGR Value
fire	Red	<code>(0, 0, 255)</code>
smoke	Orange	<code>(0, 165, 255)</code>

HUD Overlay

Rendered at bottom-left with `cv2.FONT_HERSHEY_SIMPLEX`, scale 0.45, line type `cv2.LINE_AA`.

Format: `FPS:{fps:.0f} Fire:{fire_count} Smoke:{smoke_count} Model:fire-detection`

HUD color: red if fire detected, orange if only smoke, green if nothing.

Label Format

Text drawn above each bounding box: `"{label} {confidence*100:.0f}%"` e.g. `"fire 87%"`

Black background chip sized to the text using `cv2.getTextSize()`, ensuring readability on any background.

10. Severity Scoring System

The pipeline computes a real-time severity score per frame:

`severity_score = min(100.0, (fire_count × 45) + (smoke_count × 20) + (fps × 2))`

Score Range	Label
70 – 100	<code>"critical"</code>
40 – 69	<code>"high"</code>
15 – 39	<code>"medium"</code>
0 – 14	<code>"low"</code>

Score contribution analysis:

- 1 fire detection alone → score of 45 (high)
- 2 fire detections → score of 90 (critical)
- 1 smoke only → score of 20 (medium)
- 3 smoke → score of 60 (high)
- FPS contributes a small additive bias (e.g., 30 FPS adds 60 points)

Note: The FPS term in the severity formula is anomalous — it means even with 0 detections and high FPS, the score will not be 0 (the function returns 0 only if `total_hazards == 0`, which correctly bypasses the formula).

The score is only non-zero when `total_hazards > 0`:

```
severity_score = 0.0
if total_hazards > 0:
    severity_score = min(100.0, round((fire_count * 45) + (smoke_count * 20) + (self.fps * 2), 1))
```

11. CLI Interface

Entry point: `python run.py`

```
usage: run.py [-h] [--source SOURCE] [--output OUTPUT] [--conf CONF] [--device DEVICE]
```

Standalone Fire Detection

optional arguments:

```
--source SOURCE    Path to input image or video, or 'webcam' for camera [REQUIRED]
--output OUTPUT    Path to save output (default: output.jpg)
--conf CONF        Confidence threshold, 0.0-1.0 (default: 0.25)
--device DEVICE    Inference device: cpu, cuda, or GPU index (default: cpu)
```

Supported video formats: `.mp4`, `.avi`, `.mov`, `.mkv`

Exit behavior: exits with code 1 if `--source` is not provided.

Example commands:

```
# Image
python run.py --source image.jpg --output result.jpg

# Video
python run.py --source footage.mp4 --output out.mp4 --conf 0.3

# Webcam with CUDA
python run.py --source webcam --device cuda

# GPU with higher confidence
python run.py --source image.jpg --conf 0.5 --device cuda
```

12. Library API

The `src` package can be used as a Python library:

```
import cv2
from src.pipeline import FireAIPipeline
from src.detector import FireDetector, Detection

# Full pipeline (recommended)
pipeline = FireAIPipeline(
    detector_conf=0.3,    # confidence threshold
    device="cpu",         # or "cuda"
    target_fps=30,        # used for FPS tracking window
    on_results=None,      # optional callback: fn(detections, frame)
)

frame = cv2.imread("image.jpg")
detections = pipeline.process_frame(frame)  # returns List[Detection]
annotated = pipeline.annotate_frame(frame, detections)  # returns np.ndarray
```

```

stats = pipeline.get_stats()                # returns dict (see Section 7)

# Raw detector (no annotation, no stats)
detector = FireDetector(conf_threshold=0.25, device="cpu")
detections = detector.detect(frame)

# Weight download utilities
from src.download_weights import ensure_weights, download_weights
path = ensure_weights()    # only downloads if absent
path = download_weights()  # always downloads

```

Callback Pattern

```

def on_detection(detections, frame):
    for d in detections:
        print(f"{d.label} at {d.bbox} ({d.confidence:.2f})")

pipeline = FireAIPipeline(on_results=on_detection)

```

13. Model Weight Management

Automatic Download

On first construction of `FireDetector`, if `src/weights/fire_detection_yolov8m.pt` is absent:

1. `_load_model()` calls `_download_weights()`.
2. URL resolved from: constructor `weights_url` arg → `FIRE_MODEL_HF_URL` env var → hardcoded Hugging Face URL.
3. Streamed HTTP GET with 30-second timeout, 8 KB chunks.
4. On failure: partial file deleted, `RuntimeError` raised.

Manual Download

```
python -m src.download_weights
```

Override URL

Via `.env` at project root:

```
FIRE_MODEL_HF_URL=https://your-host.com/custom_weights.pt
```

Or via environment variable:

```
set FIRE_MODEL_HF_URL=https://your-host.com/custom_weights.pt # Windows
export FIRE_MODEL_HF_URL=https://your-host.com/custom_weights.pt # Unix

```

Graceful Degradation

If weights cannot be downloaded or found, `FireDetector.available` is set to `False` and `detect()` returns an empty list — the system continues running without crashing.

.gitignore Rules for Weights

```
src/weights/
```

```
weights/  
*.pt  
*.onnx  
fire_detection_yolov8m.pt
```

The `.gitattributes` file explicitly marks weight directories as binary (`-text`) to prevent line-ending conversion:

```
src/weights/** -text  
weights/** -text
```

14. Dataset Information

Dataset Name

D-Fire — strongly inferred from the embedded training run name `yolov8n-dfire` in the checkpoint's `train_args['name']`. Not confirmed via `data.yaml` (absent from repository).

Dataset Source

Likely Kaggle — inferred from the embedded `data.yaml` path: `/Users/rbh/Downloads/archive/data.yaml`. The `/Downloads/archive/` path pattern is characteristic of a Kaggle dataset archive download on macOS.

What Is Confirmed (From Checkpoint)

Property	Value
Number of classes	2
Class labels	<code>{0: 'smoke', 1: 'fire'}</code>
Training image size	640 × 640
Dataset fraction used	1.0 (100%)

What Is Not In This Repository

- `data.yaml`
- Training image count
- Validation image count
- Test image count
- Dataset splits ratio

Hugging Face Model Source

The weights were obtained from: <https://huggingface.co/rabahdev/fire-smoke-yolov8n>

Original file: `best.pt` (renamed locally to `fire_detection_yolov8m.pt`)

15. Configuration and Environment

Logging

Configured in `run.py`:

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(name)s: %(message)s"
)
```

Each module uses its own named logger (`logging.getLogger(__name__)`). The pipeline logs at DEBUG level for per-frame timing; detector logs at INFO for model load events and at WARNING/ERROR for download failures.

Environment Variables

Variable	Default	Description
<code>FIRE_MODEL_HF_URL</code>	<code>https://huggingface.co/rabahdev/fire-smoke-yolov8n/resolve/main/best.pt</code>	Override for model weight download URL

.env File Support

`src/download_weights.py` reads a `.env` file from the project root on import. Supports `KEY=VALUE` format with `#` comments. Does not override already-set environment variables.

Git Configuration

`.gitattributes` enforces:

- Unix line endings (`eol=lf`) for all `.py`, `.md`, `.yaml`, `.yml` files
- Binary treatment (`-text`) for all files under `src/weights/` and `weights/`

16. Git History and Development Timeline

All commits authored by `vipul1029 <tovipul.kr@gmail.com>`. The entire codebase was written in a single session on **2026-08-19**.

Commit	Hash	Date	Description	Files Changed
1	<code>ccee9ce</code>	Aug 19, 02:04	chore: add git configuration	<code>.gitattributes</code> , <code>.gitignore</code>
2	<code>bd1cd82</code>	Aug 19, 02:06	docs: add project documentation	<code>README.md</code> (88 lines added)
3	<code>1885b09</code>	Aug 19, 02:06	chore: add project dependencies	<code>requirements.txt</code> (5 lines)
4	<code>48bfd36</code>	Aug 19, 02:06	feat: add application entry point	<code>run.py</code> (98 lines)
5	<code>9ca70b6</code>	Aug 19, 02:09	Update README.md	<code>README.md</code> (1 line)

Commit	Hash	Date	Description	Files Changed
6	0d12619	Aug 19, 02:18	docs: update project documentation	README.md (+251/-55 lines)
7	5234b6e	Aug 19, 02:19	feat: initialize detection module	src/__init__.py (8 lines)
8	10ca1a2	Aug 19, 02:19	feat: add fire and smoke detector	src/detector.py (174 lines)
9	2f3e850	Aug 19, 02:19	feat: add detection pipeline	src/pipeline.py (145 lines)
10	2744222	Aug 19, 02:19	feat: add model weight downloader	src/download_weights.py (46 lines)
11	611013a	Aug 19, 02:22	chore: ignore test and generated files	.gitignore (5 lines added)

Total tracked source code: ~476 lines of Python across 5 files.

17. File Inventory

Source-Controlled Files

File	Size (approx)	Lines	Purpose
src/__init__.py	~180 B	8	Package API surface
src/detector.py	~6.5 KB	175	Model + inference
src/pipeline.py	~5.5 KB	146	Orchestration + annotation
src/download_weights.py	~1.5 KB	47	Weight utilities
run.py	~3.4 KB	99	CLI entry point
requirements.txt	73 B	5	Dependencies
README.md	6.5 KB	285	User documentation
.gitignore	707 B	60	Version control exclusions
.gitattributes	130 B	8	Line ending and binary rules

Untracked / Gitignored Files Present Locally

File	Size	Description
inputvideo.mp4	~2.5 MB	Sample input video
output.mp4	~14.4 MB	Generated output video (latest run)
result.mp4	~14.4 MB	Another generated result video
output.jpg	~10.5 KB	Sample annotated image output
test_image.jpg	~5.3 KB	Sample test input image

File	Size	Description
DATASET_IDENTIFICATION_REPORT.md	14.6 KB	Deep checkpoint analysis
PRESENTATION_CONTENT_AND_SOURCES.md	18.4 KB	Presentation source material
SECTION_3_PROJECT_METHODODOLOGY.md	63.8 KB	Methodology document
Fire_Smoke_Detection_Research_Presentation.pptx	53.4 KB	Initial presentation
Fire_Smoke_Detection_Research_Presentation_FINAL.pptx	54.6 KB	Final presentation
gen_final.py	42.6 KB	Presentation generation script
generate_presentation.py	52.2 KB	Presentation generation script
.venv/	Large	Python virtual environment

18. Known Issues and Discrepancies

1. Model Filename Mismatch

The weights file is named `fire_detection_yolov8m.pt` (implying medium) but the actual architecture embedded in the checkpoint is **YOLOv8n (nano)**, confirmed by `depth_multiple=0.33`, `width_multiple=0.25`, and parameter count of ~3M.

The `README.md` incorrectly states "Architecture: YOLOv8 medium". Any research or documentation should state **YOLOv8n**.

2. Training vs. Inference Version Mismatch

The model was trained with Ultralytics `8.4.51` but the project pins inference to `8.2.18`. This is a minor version gap and unlikely to cause issues but could theoretically affect certain model behaviors.

3. FPS Term in Severity Score

The severity score formula includes `self.fps * 2`. At 30 FPS this adds 60 points to the score — which is significant. However, severity is only computed when `total_hazards > 0`, so this does not cause false alarms with zero detections. It does mean the same detection count at higher FPS will produce a higher severity score, which may not reflect actual risk.

4. `total_potholes: 0` in `get_stats()`

The stats dict includes `"total_potholes": 0` — a compatibility field for a shared API with a pothole detection pipeline. This has no functional impact on this module.

5. Webcam Frame Retrieval vs. Video Frame Retrieval

`process_image` and `process_webcam` use `cap.read()` (single-step). `process_video` uses `cap.grab()` + `cap.retrieve()` (two-step). Both are correct for their use cases, but the inconsistency is worth noting.

6. Target FPS Not Enforced

`target_fps` is stored and used for the rolling FPS window size, and exposed in `info` and `get_stats()`, but **it does not actually throttle the processing rate**. Frames are processed as fast as possible regardless of `target_fps`. The parameter is effectively a metadata field.

7. No Thread Safety

`FireAIPipeline` maintains mutable state (`frame_idx`, `_fps_times`, `_latest_detections`) without locks. This is safe for single-threaded use but would produce race conditions if `process_frame()` and `get_stats()` were called from different threads concurrently.

19. Summary of Verified Facts

Category	Fact	Source
Architecture	YOLOv8n (nano), NOT medium	Checkpoint <code>model.yaml</code>
Parameters	3,011,238	<code>model.info()</code>
Layers	226	<code>model.info()</code>
Classes	2: smoke (0), fire (1)	<code>model.names</code> , <code>detector.py:70</code>
Input size	640 × 640 px	<code>train_args['imgsz']</code>
Training epochs	50	<code>train_args['epochs']</code>
Batch size	16	<code>train_args['batch']</code>
Optimizer	auto	<code>train_args['optimizer']</code>
Initial LR	0.01	<code>train_args['lr0']</code>
Momentum	0.937	<code>train_args['momentum']</code>
Weight decay	0.0005	<code>train_args['weight_decay']</code>
Early stopping	15 epochs patience	<code>train_args['patience']</code>
Training device	Apple Silicon (mps)	<code>train_args['device']</code>
Training date	2026-05-20	<code>ckpt['date']</code>
Training IoU	0.7	<code>train_args['iou']</code>
Inference conf	0.25 (default)	<code>detector.py:57</code>
Inference IoU	0.45	<code>detector.py:58</code>
mAP@0.5 (final)	0.76498 (76.5%)	<code>train_metrics</code>
mAP@0.5:0.95 (final)	0.44460 (44.5%)	<code>train_metrics</code>
Precision (final)	0.77529 (77.5%)	<code>train_metrics</code>
Recall (final)	0.69452 (69.5%)	<code>train_metrics</code>
Dataset	D-Fire (strongly inferred)	run name <code>yolov8n-dfire</code>

Category	Fact	Source
Dataset source	Likely Kaggle (inferred)	path /Downloads/archive/
Training framework	Ultralytics 8.4.51	ckpt['version']
Inference framework	Ultralytics 8.2.18	requirements.txt
AMP training	Enabled	train_args['amp']
Transfer learning	Yes (pretrained)	train_args['pretrained']
Weight host	Hugging Face rabahdev/fire-smoke-yolov8n	detector.py:66
Project type	Real-time fire and smoke detection system	run.py, src/
Supported inputs	Images, MP4/AVI/MOV/MKV videos, webcam	run.py:94
Python requirement	3.9+	README
OS support	Windows, macOS, Linux	README

This report was generated on 2026-09-11 by deep static analysis of all source files, git history, the .pt model checkpoint metadata (as documented in DATASET_IDENTIFICATION_REPORT.md), and the README.